Making Stack Comments Checkable: Static Type Checking for Forth

Jaanus Pöial
Tallinn University of Technology

Abstract

Forth programmers already document words with stack-effect comments. This paper shows how nearly the same notation can drive a static checker. An effect records the required and produced stack suffix, while indexed type names retain the identity of values moved by words such as `DUP`, `SWAP`, and `OVER`. Three operations form the useful core: join adjacent effects for straight-line code, find a common contract for alternatives, and compare one with two executions to approximate repetition. A native Gforth prototype loads its type hierarchy, word effects, parser behavior, and control structures from text files. It infers effects for colon definitions and reports stack or type clashes without running the program. The presentation emphasizes what a Forth user can write, see, and rely on; formal proofs and deeper algebra are left to the cited technical work.

Keywords: Forth; stack effects; static checking; symbolic execution; concatenative languages.

1 From comments to contracts

A stack comment is one of Forth's best documentation tools:

```
: SQUARED  DUP * ;        ( n -- n )
```

The comment states a calling convention, but a normal Forth system neither checks it nor derives it. A later edit can leave the comment wrong. The problem becomes harder around stack shuffles, branches, and loops, where stack height alone is not enough.

The checker described here treats an effect as a small symbolic execution of a word. As in ordinary Forth notation, the top of stack is on the right:

```
+       ( n n -- n )
@       ( a-addr -- x )
C@      ( c-addr -- char )
```

The effect constrains only the part of the data stack touched by the word. An unmentioned prefix passes through unchanged. Consequently, the checker need not know the complete stack at every point: it can infer which suffix must have been present before a phrase.

This is a source-level checker, not a modified Forth virtual machine. It reads source text and computes effects instead of executing the words. Its job is to answer practical questions:

- Does every consumer receive enough stack items of compatible types?
- Which input value reaches each output after stack shuffling?
- Do alternatives admit one usable stack contract?
- Does a repeated phrase appear stack-stable under the implemented test?

The approach does not make Forth dynamically typed at run time, nor does it attach tags to cells. Types exist only in the checker and may be as broad or as detailed as the selected application profile. The calculus follows earlier work on algebraic and multiple stack effects, typed wildcards, must-analysis, and executable checking tools [2, 3, 4, 5, 6].

2 Effects that remember values

2.1 Types supplied by the application

The checker loads a finite subtype relation. A typical Forth-oriented fragment is:

```
type X cell x
type n number N
type char c
type flag boolean f
type addr a
type c-addr ca
type a-addr aa

rel n < X
rel addr < X
rel char < n
rel c-addr < addr
rel a-addr < c-addr
```

Here `X`, `cell`, and `x` are aliases. A `a-addr` is acceptable where a `c-addr`, `addr`, or `X` is required, but the converse is not necessarily safe. The checker keeps the more specific type when two compatible boundaries meet. It reports a clash when neither type is a subtype of the other.

The hierarchy is deliberately profile-dependent. For example, one profile may place `flag` below `n`, while a stricter Forth-2012 profile [1] keeps flags separate from numeric types. This makes assumptions visible instead of embedding one machine model in the analyzer.

2.2 Indexed symbols

Types alone lose important information. The two occurrences produced by `DUP` are not unrelated values, and `SWAP` does not manufacture new values. Indexed symbols state these correlations:

```
DUP   ( X[1] -- X[1] X[1] )
SWAP  ( X[2] X[1] -- X[1] X[2] )
OVER  ( X[2] X[1] -- X[2] X[1] X[2] )
```

An index is local to an effect declaration. Repeated `[1]` means the same symbolic input has flowed to those positions. Two plain, unindexed `X` occurrences are fresh values; they are not silently assumed equal. Before checking a phrase, the implementation renames local indices so effects from different words cannot collide.

Indices are useful even on an untyped cell machine. They let a use later in a phrase refine an earlier value and carry that information through intervening stack words. They express data flow without introducing variable names into the Forth source.

3 The effect calculus in working terms

Only three combining operations are needed for the supported control forms. Table 1 gives their programmer-facing interpretation.

Source form	Operation	Question answered
`P Q`	composition	Can the output of `P` supply the input of `Q`?
branches	common contract	What stack effect is valid for either alternative?
repetition	one-versus-two test	Is the repeated phrase stable in this local approximation?

Table 1: The three core effect operations.

3.1 Sequence: match the touching boundaries

For adjacent phrases, the checker matches the top output of the first effect against the top input of the second, then works inward. Compatible symbols are replaced everywhere by one symbol with the more precise type. Any unmatched requirement becomes part of the inferred input; any unmatched result becomes part of the output.

For example, integer literals, `+`, and `DUP` may be declared as:

```
literal integer ( -- n )
+               ( n n -- n )
DUP             ( X[1] -- X[1] X[1] )
```

The phrase

```
1 2 + DUP
```

first produces two numbers, consumes them with `+`, and then matches the resulting `n` against `DUP`'s general `X`. The inferred result is:

```
( -- n[1] n[1] )
```

Both output cells come from the one numeric result. Substitution across the whole working phrase is the key: refinement discovered at one boundary remains visible on every correlated occurrence.

A mismatch is equally direct. Given `C@ ( c-addr - char )` and `! ( X a-addr - )`, the phrase `C@ !` attempts to use the produced `char` as the address consumed by `!`. If those types are incomparable in the selected profile, checking stops at that boundary rather than waiting for a bad memory access.

3.2 Alternatives: find one honest contract

An `IF` has more than one path, but callers need one effect for the whole definition. The checker therefore combines branch effects only when it can construct a common contract. It aligns their visible inputs and outputs, accounts for a preserved stack prefix when depths differ, and unifies corresponding types and identities. The condition's `flag` consumption is then composed with that contract.

The easy case is two branches with the same behavior:

```
: KEEPIF  IF DUP ELSE DUP THEN ;
```

The inferred effect is:

```
KEEPIF  ( X[1] flag -- X[1] X[1] )
```

By contrast, one branch leaving an extra item while the other removes one will normally have no common usable boundary and is rejected. Differences in type can still be accepted when one type is a declared subtype of the other; the result records the stronger requirement that is valid for both paths.

This operation is sometimes called a greatest lower bound in the effect literature. For day-to-day use, "common branch contract" is the important idea. It is not a union of everything either branch might do. It keeps only a single symbolic interface that the checker can justify for both.

3.3 Loops: compare one pass with two

A loop body can execute an unknown number of times. The prototype uses a finite, deliberately modest test: compute the effect of one pass, compute the effect of two consecutive passes, and ask the branch-contract operation for a common stable summary. This catches common mistakes such as a body that grows or shrinks the data stack on each iteration.

For example, including the terminating test in the repeated phrase makes this loop stack-stable:

```
: ULOOP  BEGIN DUP 0= UNTIL ;
```

With `0= ( n - flag )` and `UNTIL ( flag - )`, its inferred effect is:

```
ULOOP  ( n[1] -- n[1] )
```

The number survives each pass; the duplicated copy is consumed by the test. The specification language can similarly describe `BEGIN-WHILE-REPEAT` and `DO-LOOP` skeletons.

The limitation matters: agreement between one and two passes is a local approximation, not a general fixed-point computation and not a proof for every iteration count. Acceptance says that the phrase passed this stack-effect test under its primitive declarations. It does not establish termination or all semantic loop invariants.

4 Turning Forth source into effects

The algebra alone cannot read real source. Forth words may consume the next name, scan to a delimiter, change compilation state, or introduce dictionary entries. The prototype therefore separates three inputs:

1. a *type file* containing names, aliases, and subtype edges;
2. a *specification file* containing word effects and parsing roles;
3. the Forth *program file* to check.

Representative specifications are:

```
"("      parse until ")"       ( -- )
:        parse word define colon ( -- )
;        control end             ( -- )
literal integer                  ( -- n )
IF       control if              ( flag -- )
@                                  ( a-addr -- X )
```

Control syntax can also be declared as a pattern with captured source segments, followed by an effect recipe using sequencing, alternatives, and repetition. This keeps parser conventions out of the calculus and permits custom Forth-like profiles.

A colon definition is checked when it closes, and its inferred effect is added to the analysis dictionary for later source. Ordinary comments are consumed but are not trusted as declarations. Thus this source cannot force an incorrect result merely by claiming one:

```
: BAD  C@ ! ;   ( -- n )
```

Recognized locals syntax may provide a provisional shape, but a normal stack comment remains documentation.

The bundled launchers run either the native Gforth checker or the Java reference. For example:

```
./run-evaluator-gforth.sh real

./run-evaluator-gforth.sh \
  --types forth2012types.txt \
  --specs forth2012specs.txt \
  --prog forth2012prog.txt
```

On Windows, `run-evaluator-gforth.bat` provides the corresponding entry point. The native implementation itself is the single file `gforth-evaluator.fs`.

5 A complete small example

The bundled `real` profile analyzes these definitions and a top-level phrase:

```
: PEEK2   DUP @ SWAP @ ;
: MAYSWAP IF SWAP THEN ;
: NLOOP   BEGIN DUP 0= WHILE REPEAT ;
: ULOOP   BEGIN DUP 0= UNTIL ;
: KEEPIF  IF DUP ELSE DUP THEN ;

DP DP C@ 0= KEEPIF
```

The generated log includes:

```
PEEK2   ( a-addr[1] -- X[2] X )
MAYSWAP ( X[1] X[1] flag -- X[1] X[1] )
NLOOP   ( n[1] -- n[1] )
ULOOP   ( n[1] -- n[1] )
KEEPIF  ( X[1] flag -- X[1] X[1] )
```

Several details are worth reading as a Forth programmer. `PEEK2` requires an address because its input eventually reaches `@`; the type requirement travels backward through `DUP` and `SWAP`. `MAYSWAP` can promise one output order only under the stronger symbolic condition shown by its repeated index. Both loop examples preserve their numeric input according to the one-versus-two rule. `KEEPIF` consumes a flag and duplicates the remaining cell regardless of the selected branch.

For the final phrase, the checker reports the per-word refined effects and ends with:

```
DP      ( -- a-addr[1] )
DP      ( -- a-addr )
C@      ( a-addr -- char )
0=      ( char -- flag )
KEEPIF  ( a-addr[1] flag -- a-addr[1] a-addr[1] )
< a-addr[1] a-addr[1]
```

The second `DP` result is specific enough for `C@`; `0=` produces the flag consumed by `KEEPIF`; and the first address is the value duplicated on exit. This is more information than a stack-depth pass would retain, yet the result still looks like familiar Forth notation.

6 What acceptance does—and does not—mean

The checker is useful as an early consistency check, especially for legacy code whose stack comments are incomplete. It can expose underflow-like shape errors, incompatible address and scalar uses, branch interfaces that disagree, and many loop bodies with changing stack shape. External profiles also make it possible to tighten a project gradually instead of imposing one universal Forth type system.

Its guarantees are bounded by the model:

- Primitive effects are trusted. A wrong specification can make a wrong program appear valid.
- The current type match handles equal or directly comparable types; it does not search the hierarchy for an arbitrary third common subtype.
- Internal normalization can discard some automatically introduced identity correlations, reducing precision in later checks.
- Branch merging produces one conservative syntactic contract, not a set of all possible branch effects.
- The loop rule is the one-versus-two approximation described above.
- Return-stack tricks, unrestricted `EXECUTE`, non-local `THROW`, self-modifying definitions, and other metaprogramming may need special models or remain outside the supported subset.

The tool should therefore complement tests, reviews, and target-system checks, not replace them. It is best viewed as executable stack-effect documentation with useful static diagnostics.

7 Conclusion

Forth already has the right surface notation for a lightweight static analysis. Making stack comments executable requires two additions: a project-specific type relation and indices that preserve symbolic value flow. From there, three operations cover much useful source: compose effects for a word sequence, form a common contract for alternatives, and compare one with two passes for repetition.

The result remains recognizably Forth. Word effects live in text files, custom parser and control behavior can be declared, definitions receive inferred stack comments, and diagnostics point to incompatible boundaries. The method does not solve arbitrary Forth metaprogramming or general loop verification, but it offers a practical path from informal comments to machine-checked stack discipline.

References

[1] Forth 200x Standardisation Committee. *Forth 2012 Standard*, draft proposed standard, 2014. `https://forth-standard.org/standard/intro`.

[2] J. Pöial. Algebraic specifications of stack effects for Forth programs. In *1990 FORML Conference Proceedings, EuroFORML'90*, pp. 282–290, 1991.

[3] J. Pöial. Multiple stack-effects of Forth programs. In *1991 FORML Conference Proceedings, euroFORML'91*, pp. 400–406, 1992.

[4] J. Pöial. Stack effect calculus with typed wildcards, polymorphism and inheritance. Abstract and presentation, 18th EuroForth Conference, 2002.

[5] J. Pöial. Typing tools for typeless stack languages. In *Proceedings of the 22nd EuroForth Conference*, pp. 40–46, 2006.

[6] J. Pöial. Java framework for static analysis of Forth programs. In *Proceedings of the 24th EuroForth Conference*, pp. 20–23, 2008.